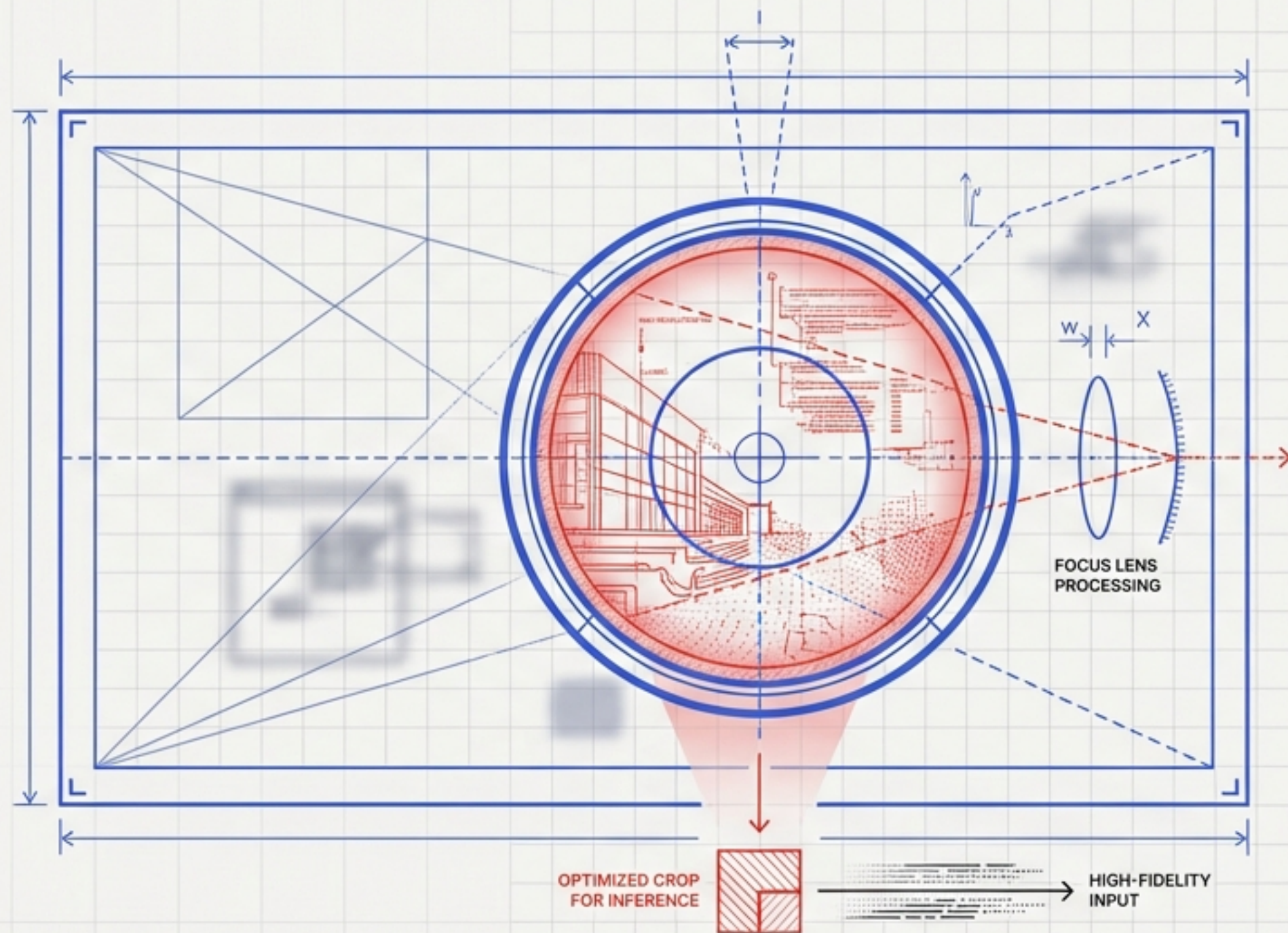


Sistema Focus Lens: Optimización de Inferencia Basada en Recortes

Aumentando la resolución efectiva y reduciendo el ruido en visión por computadora.



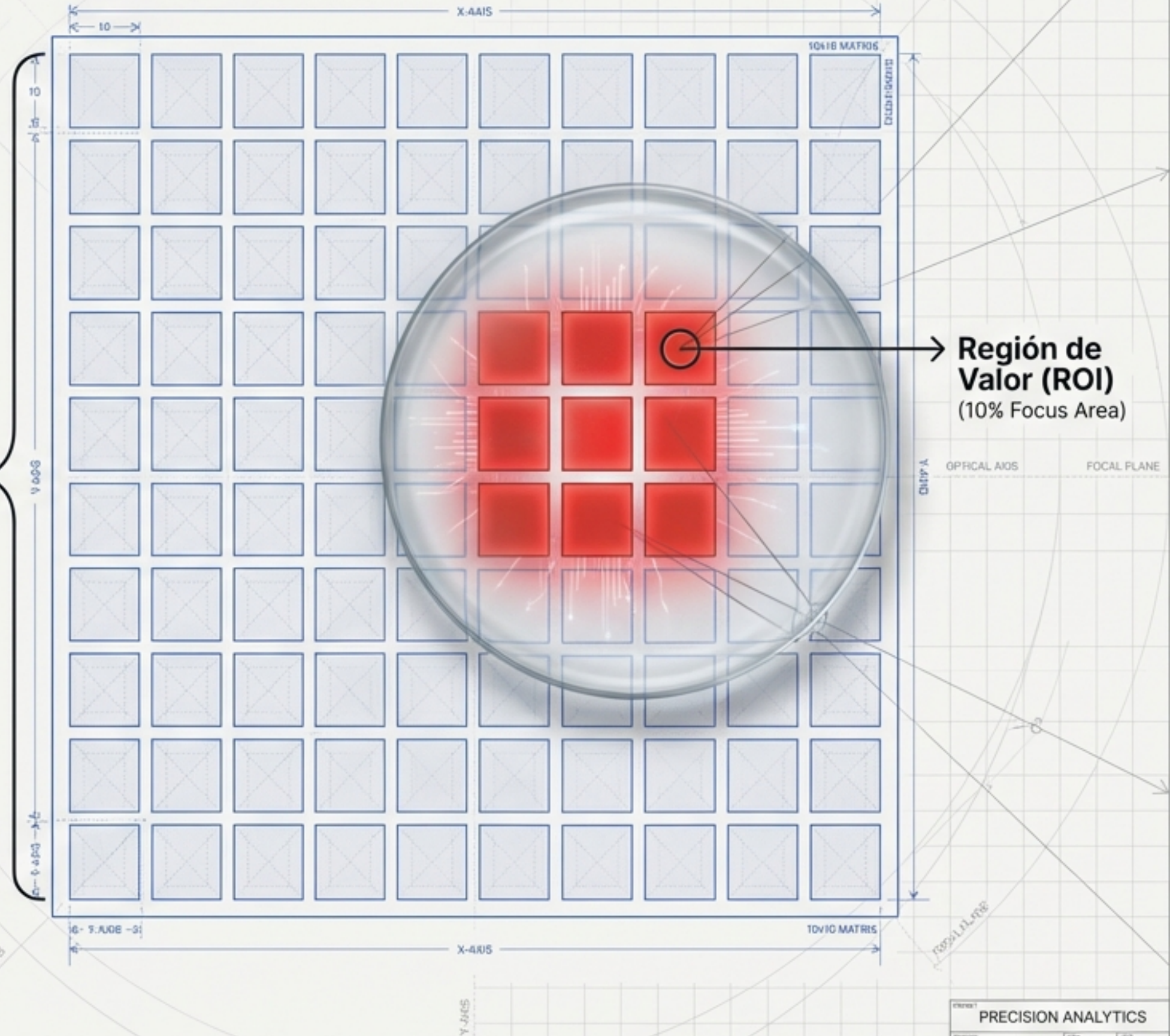
El Desafío: Escala, Ruido y Desperdicio de Cómputo

El Problema: Cuando el sujeto de interés ocupa una pequeña porción del cuadro, la mayor parte de la capacidad de inferencia del modelo se desperdicia procesando fondo irrelevante.

- **Baja Resolución Efectiva:** El modelo "ve" pocos píxeles del objeto real.
- **Ruido de Fondo:** Elementos distractores reducen la precisión de la detección.
- **Ineficiencia:** Procesamiento innecesario de áreas vacías.

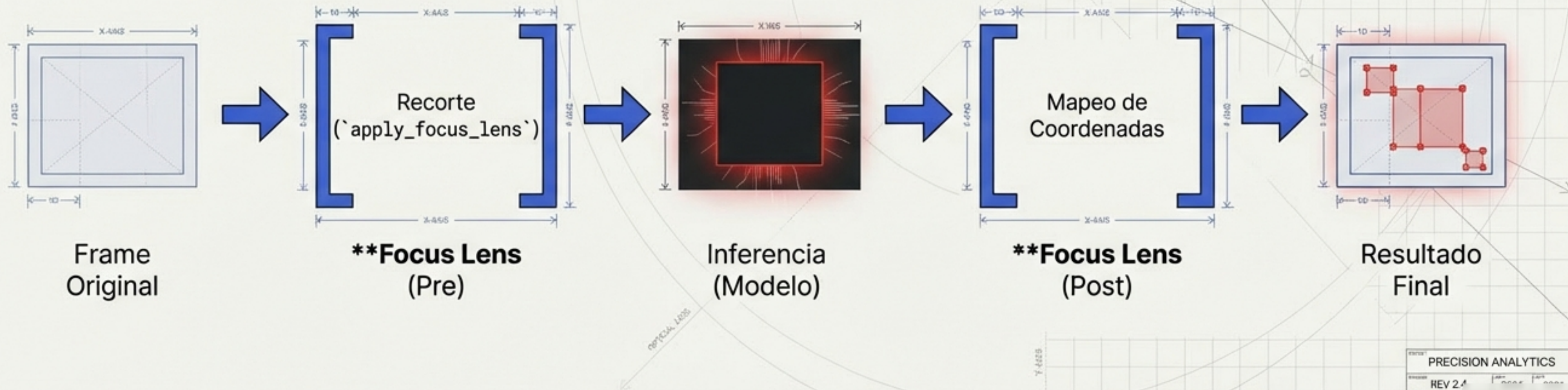
Ruido de Fondo / Desperdicio
(90% of Compute Capacity)

Waste Ratio

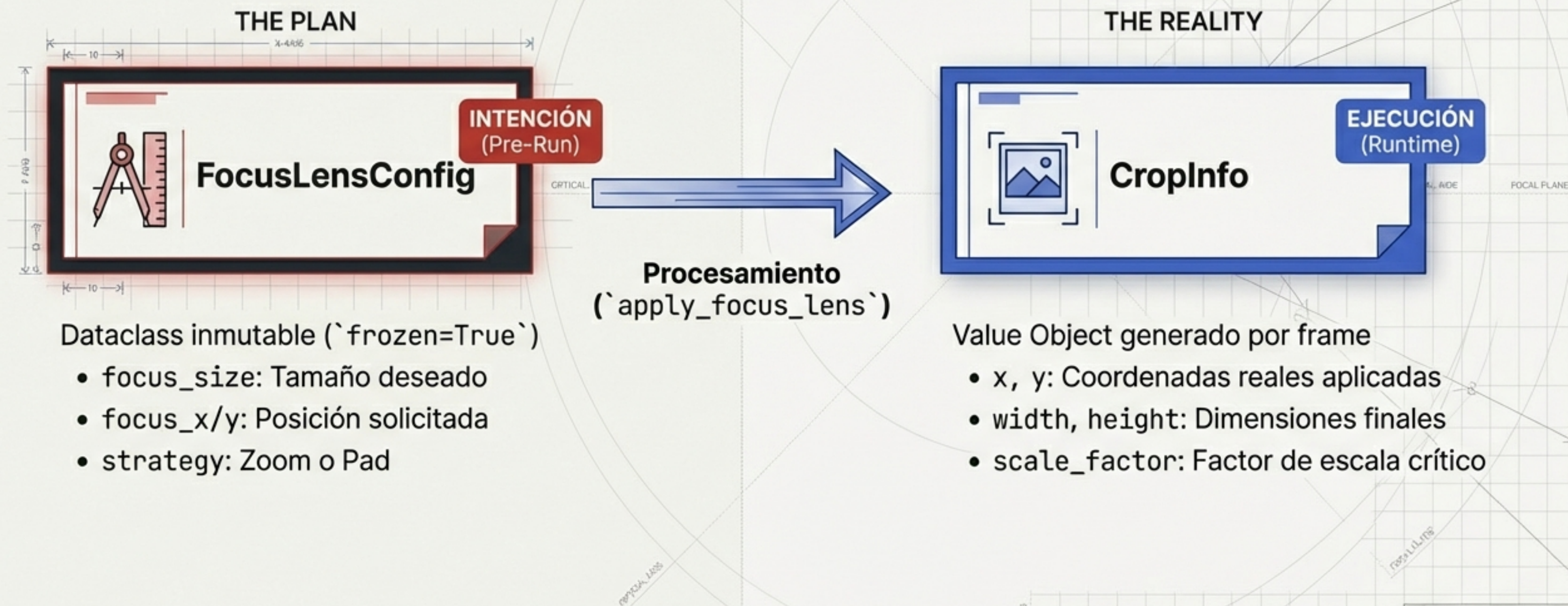


Arquitectura: Un 'Wrapper' de Pre y Post-Procesamiento

El sistema actúa como una capa intermedia alrededor del pipeline de inferencia, mejorando la precisión sin re-entrenar los modelos existentes.

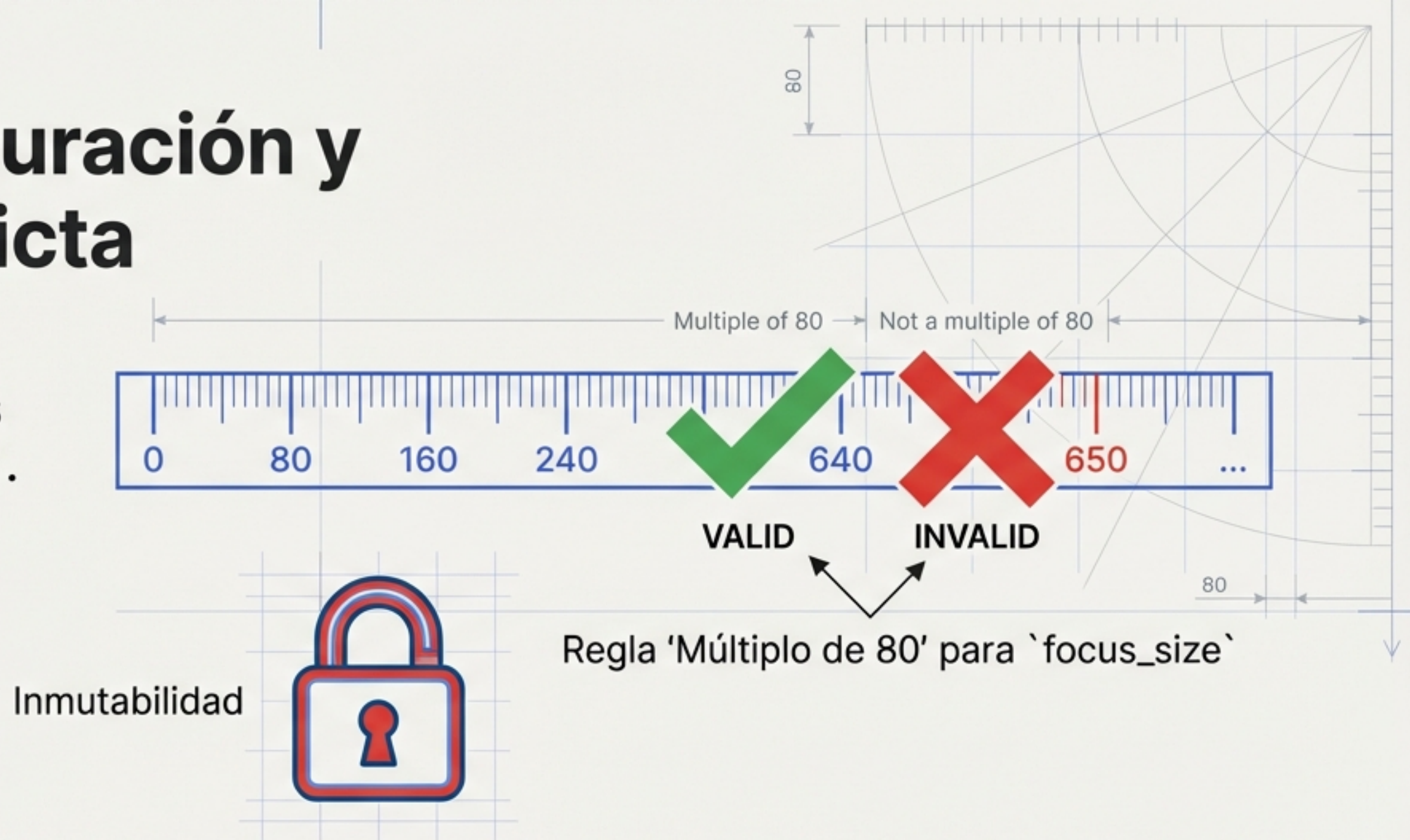


Entidades Core: La Intención vs. La Ejecución



Reglas de Configuración y Validación Estricta

La integridad del sistema se asegura mediante validaciones automáticas en ``__post_init__``.

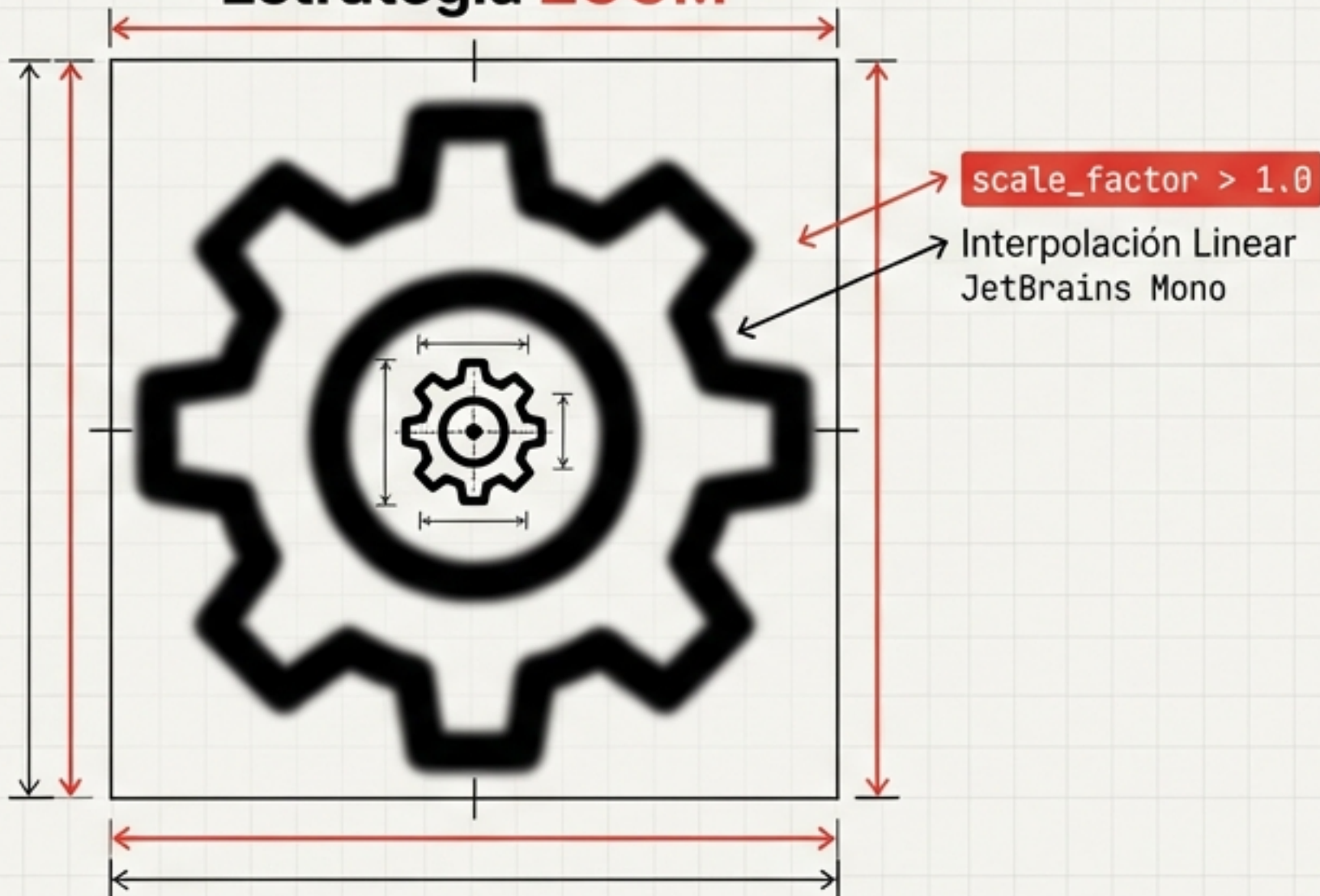


1. **Alineación con Arquitectura YOLO:** ``focus_size`` debe ser múltiplo de 80. Esto evita errores de dimensión causados por los 'strides' (pasos) de la red neuronal.
2. **Positividad y Límites:** Coordenadas no negativas y tamaño estrictamente positivo.
3. **Inmutabilidad:** La configuración se congela (``frozen=True``) para garantizar consistencia total a lo largo del pipeline.

Estrategias para Frames Pequeños

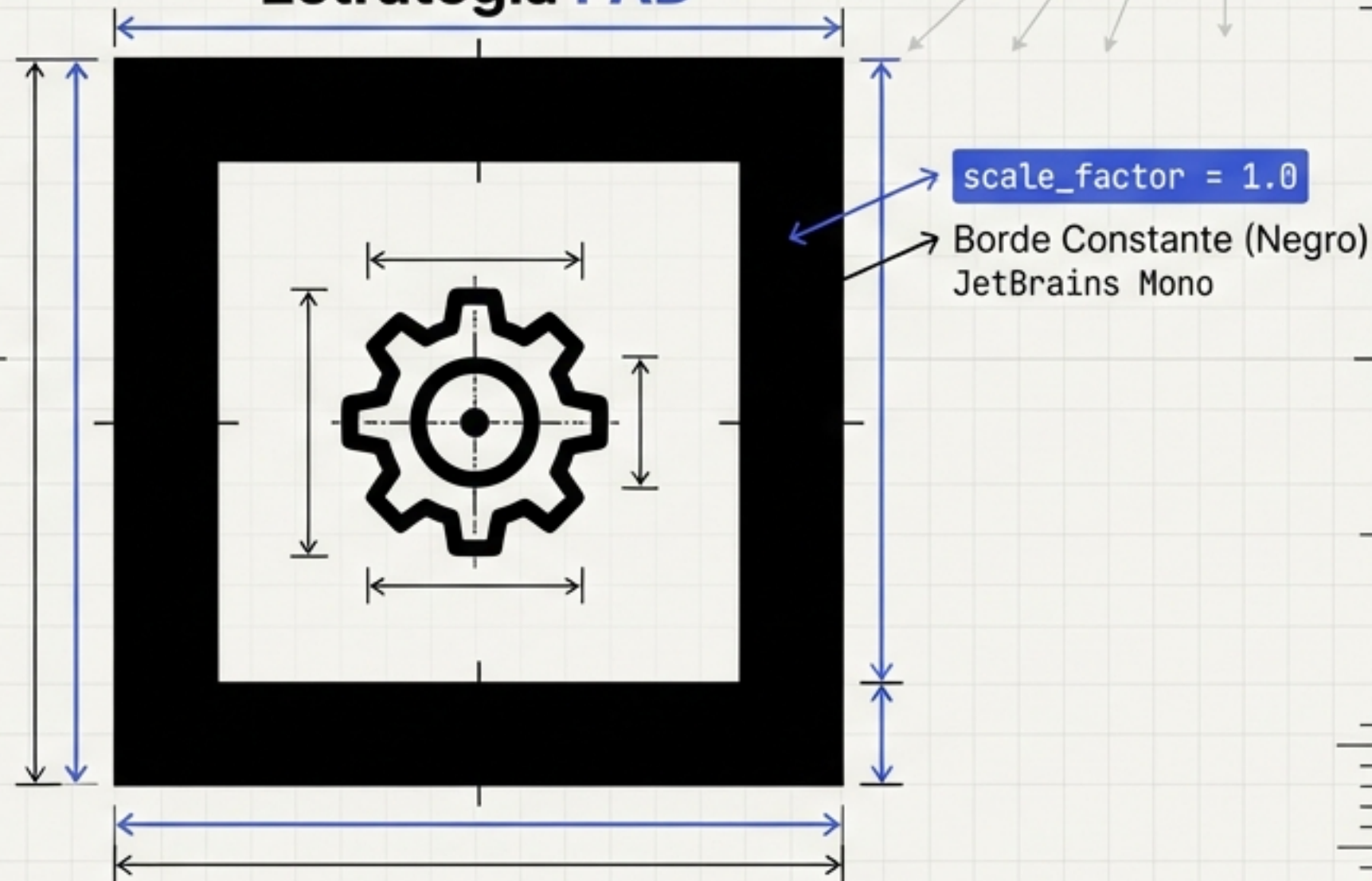
¿Qué sucede cuando el frame es más pequeño que el recorte deseado?

Estrategia **ZOOM**



ZOOM (Default): Maximiza el área de inferencia a costa de introducir artefactos de interpolación.

Estrategia **PAD**



PAD: Preserva la integridad de los píxeles originales pero sacrifica área útil de detección.

Operación 1: Aplicación del Lente (`apply\_focus\_lens`)

1. ****Cálculo de Posición:****

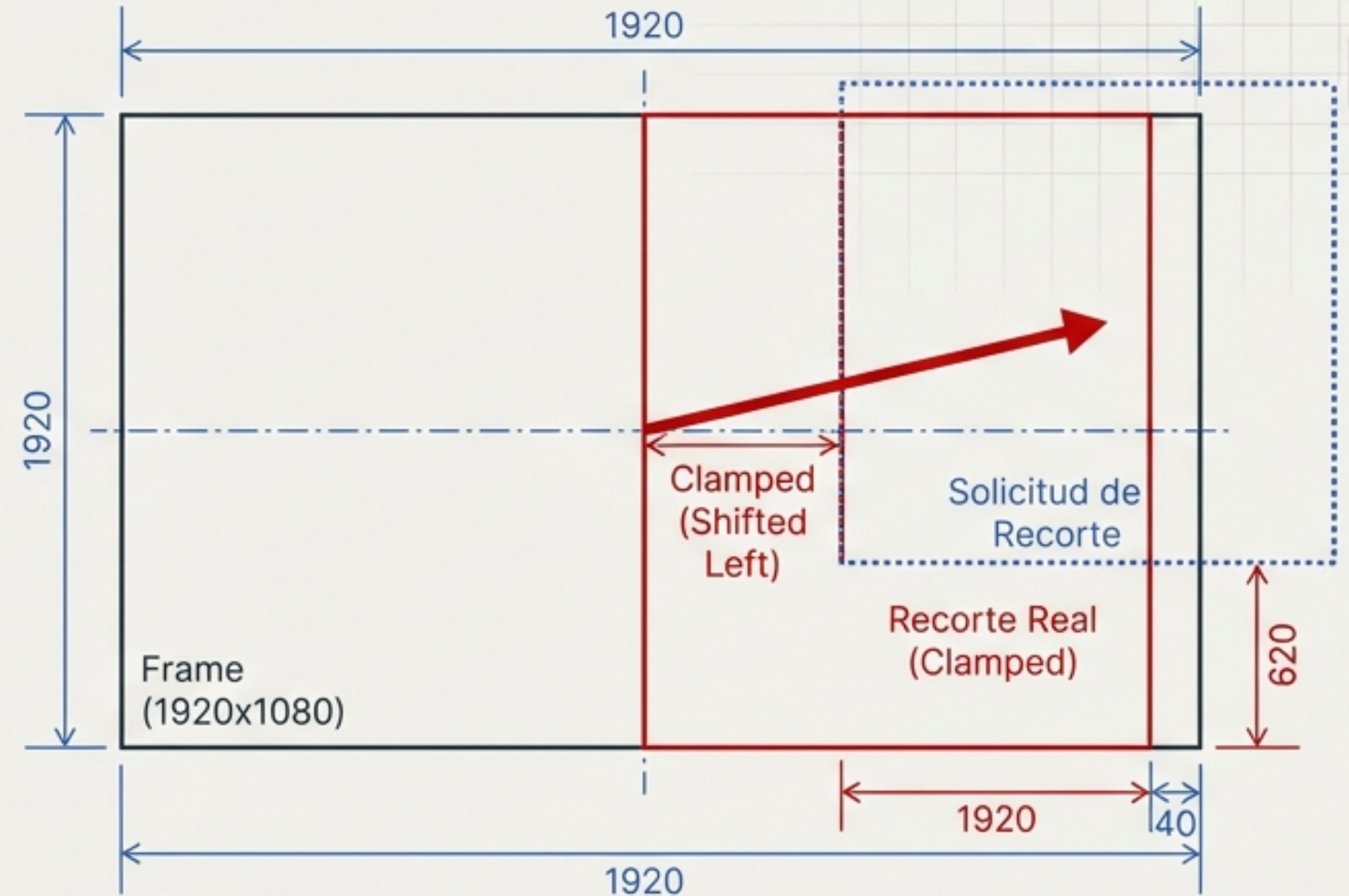
- Si `focus\_x/y` son `None`, se calcula el centro geométrico:
$$\text{(frame_dim - focus_size) // 2}$$

2. ****Clamping (Seguridad de Bordes):****

- Si la posición solicitada excede los límites, el sistema la ajusta automáticamente al borde más cercano.

3. ****Extracción y Metadata:****

- Se genera el recorte de píxeles y se instancia el objeto `CropInfo` con la posición final "clamped".

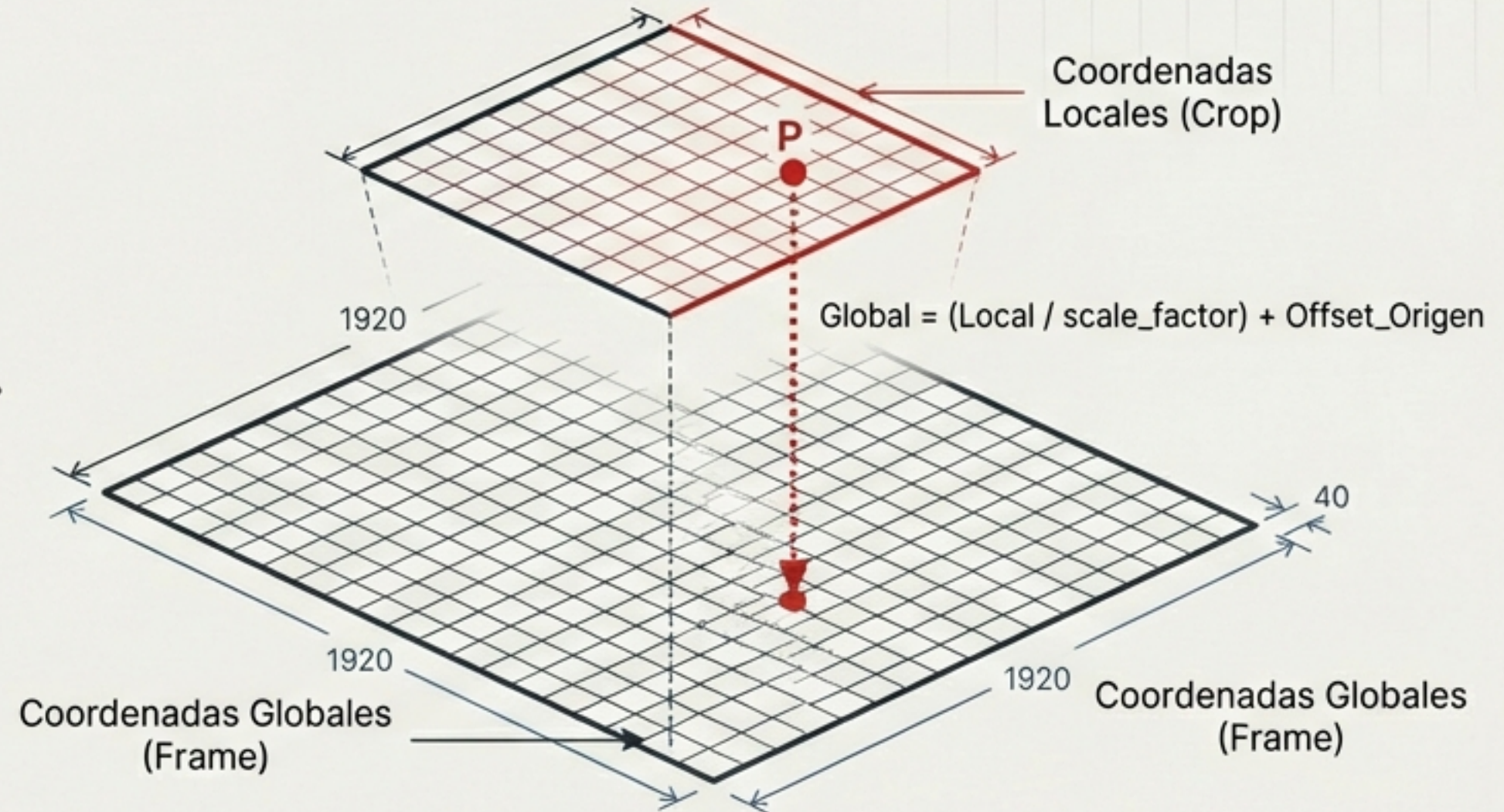


Operación 2: El Retorno (Transformación de Coordenadas)

$$\text{Global} = (\text{Local} / \text{scale_factor}) + \text{Offset_Origen}$$

La Lógica: La inferencia ocurre en un mundo "recortado" y posiblemente escalado. Para que los datos sean útiles, debemos traducir cada detección de vuelta al sistema de coordenadas original.

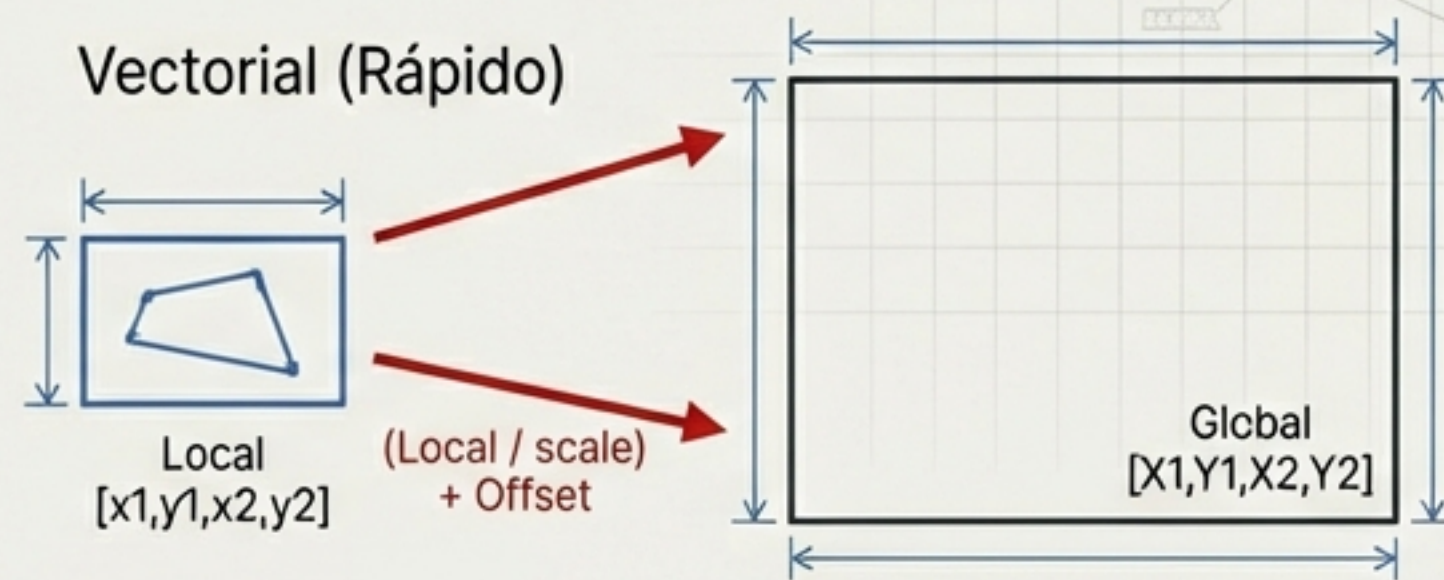
1. ****Des-escalado:** Revertir el zoom (división).
2. ****Offset:** Sumar la posición `(x, y)` donde se tomó el recorte.



Mapeo de Detecciones: Cajas y Máscaras

Bounding Boxes (Cajas)

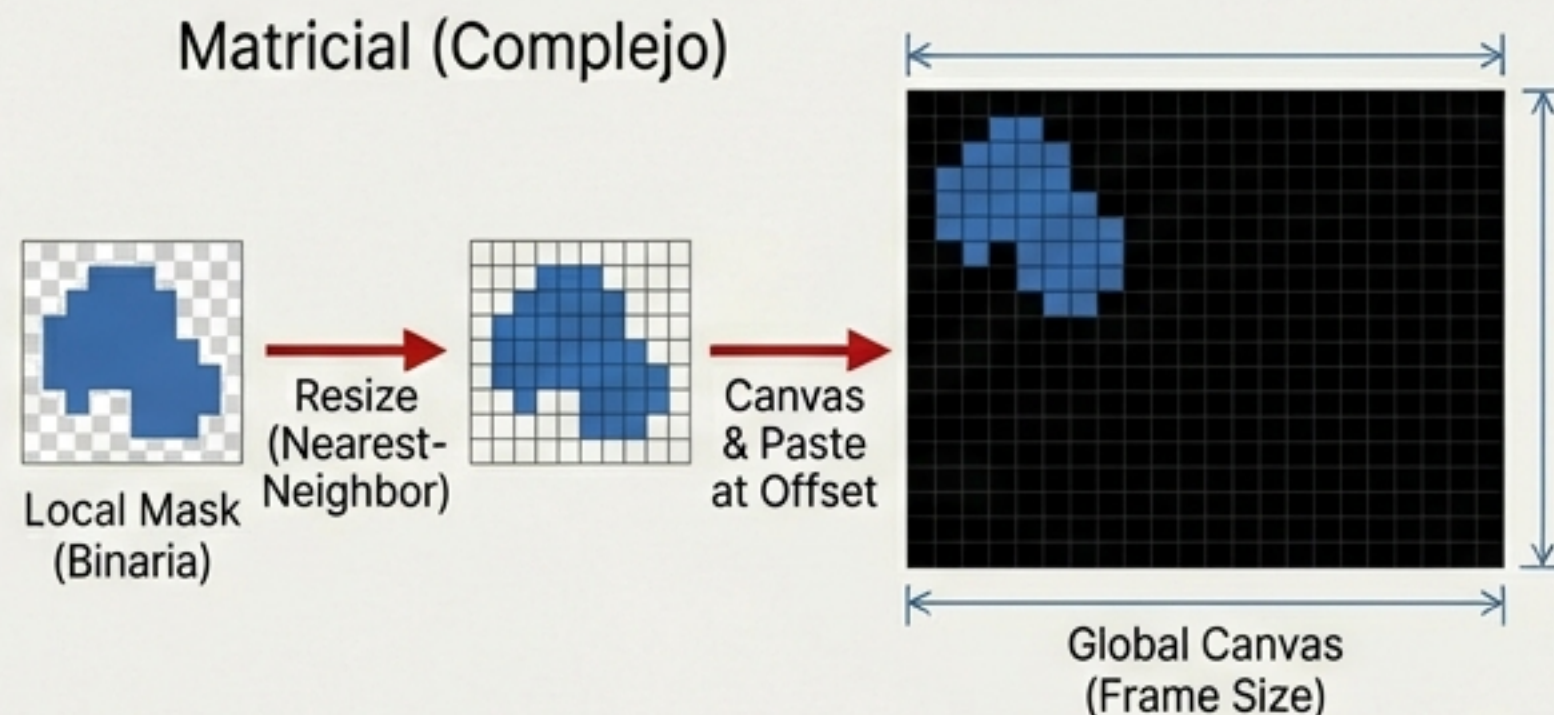
- Transformación vectorial directa usando NumPy.
- Se aplica la fórmula de des-escalado + offset a los puntos esquina $(x1, y1)$ y $(x2, y2)$ simultáneamente.



Segmentation Masks (Máscaras)

Proceso de manipulación de matrices 2D:

1. **Resize:** Redimensionar máscara binaria (Interpolación Nearest-Neighbor).
2. **Canvas:** Crear una 'lona' negra del tamaño del frame original.
3. **Paste:** Pegar la máscara redimensionada en la posición del offset.

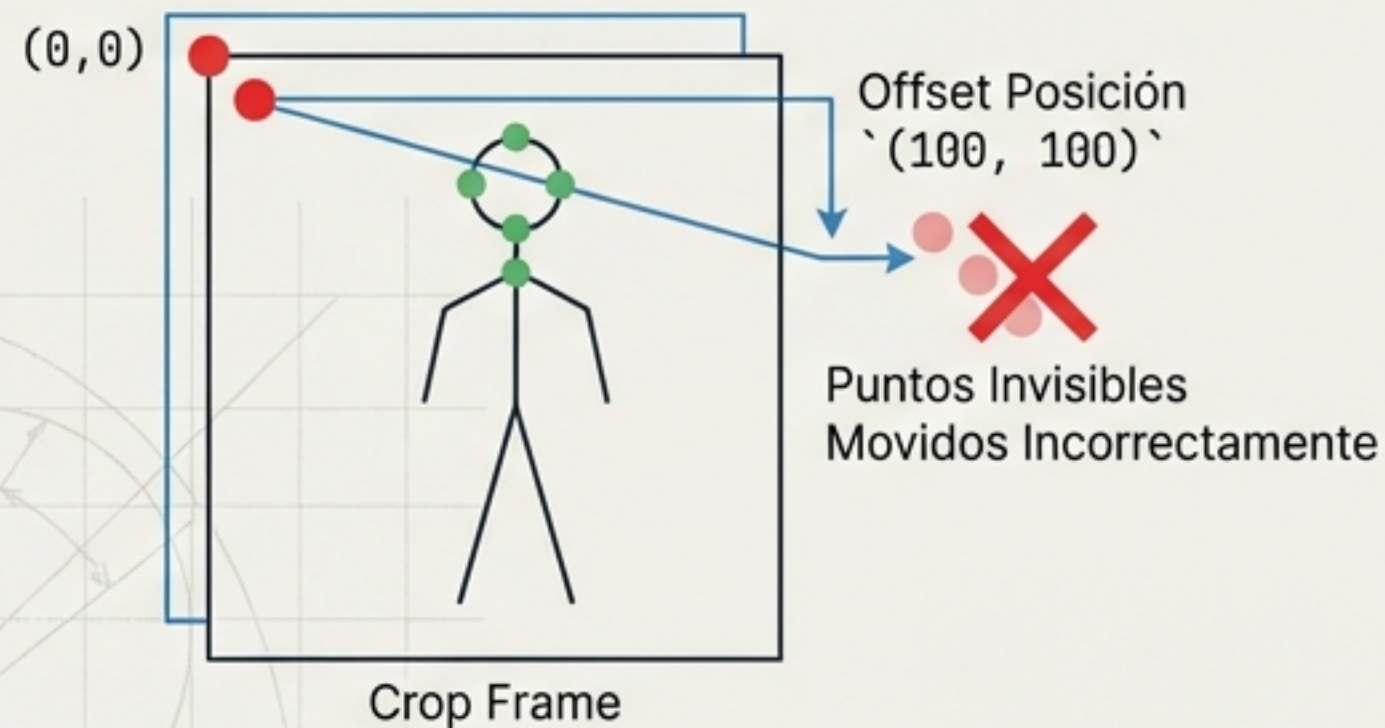


El Caso Especial de los Keypoints Invisibles

Preservando la semántica de los datos en modelos de pose.

El Problema: Los modelos marcan puntos no visibles como $(0, 0)$. Si aplicamos el offset matemático a estos puntos, los moveríamos erróneamente dentro del frame.

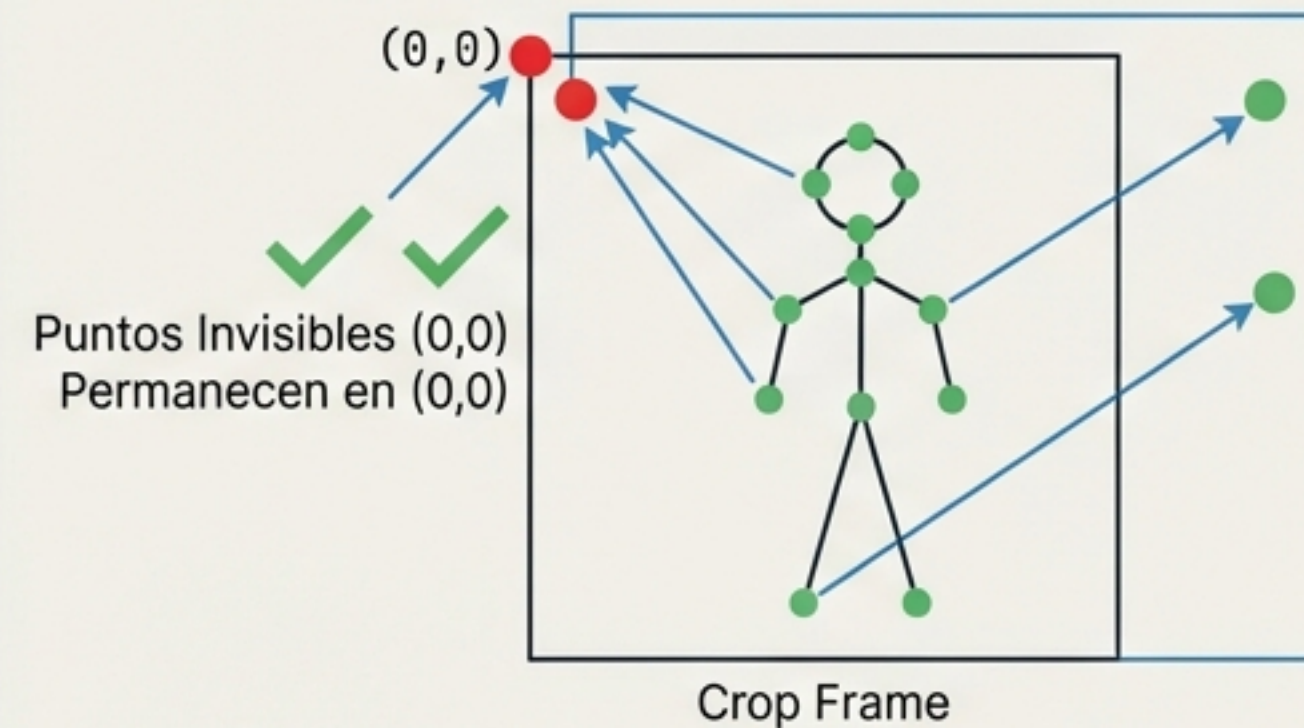
ADVERTENCIA: Transformación Errada



La Regla de Oro: Si punto $== (0,0) \rightarrow$ **NO TOCAR.**

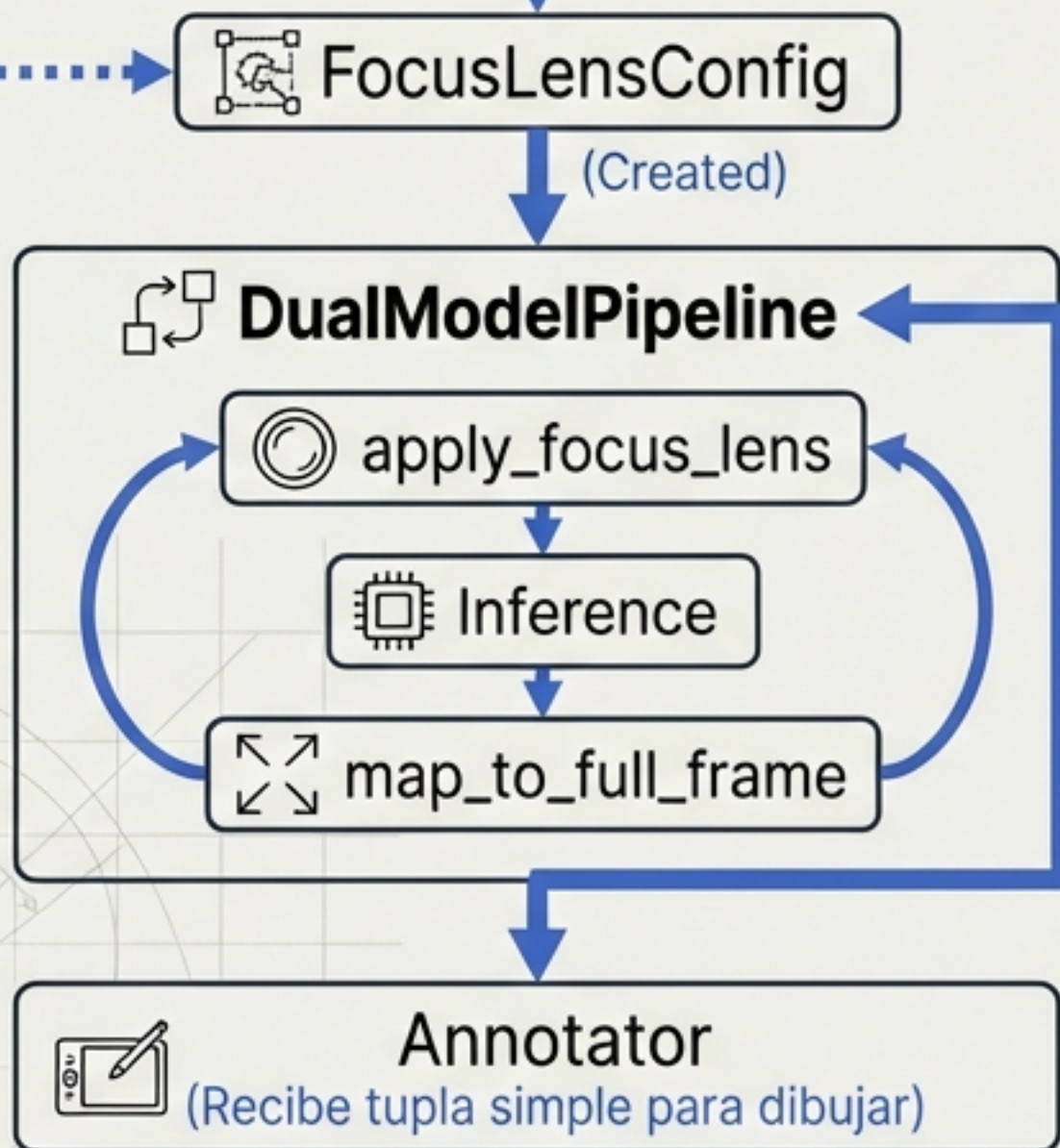
Solución Técnica: Enmascaramiento vectorial para aplicar la transformación $(\text{punto} / \text{scale}) + \text{offset}$ *solo** a los puntos visibles.

CORRECTO: Preservar $(0,0)$



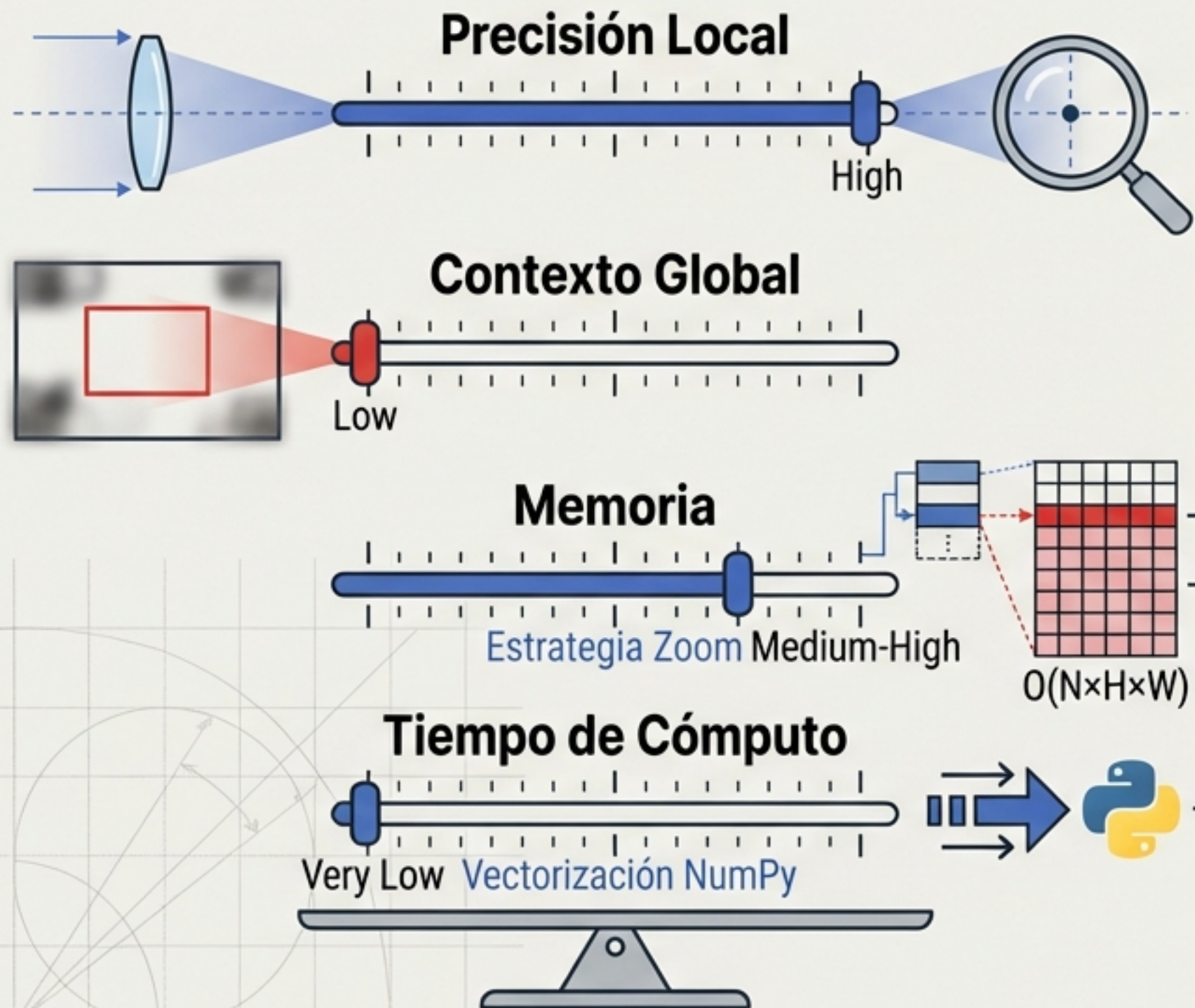
Integración en el Pipeline Dual (Luna)

```
> run_luna.py --focus-size 640 --focus-x 100
```



1. **CLI Arguments:** El usuario define parámetros al iniciar `run_luna.py`.
2. **Inicialización:** Se crea la configuración inmutable.
3. **Ciclo de Inferencia:** El pipeline invoca el recorte antes del modelo y el mapeo después.
4. **Anotación:** **CropInfo** se convierte a una tupla simple (**crop_info_to_tuple**) para compatibilidad con el sistema de dibujado.

Consideraciones de Rendimiento y Recursos



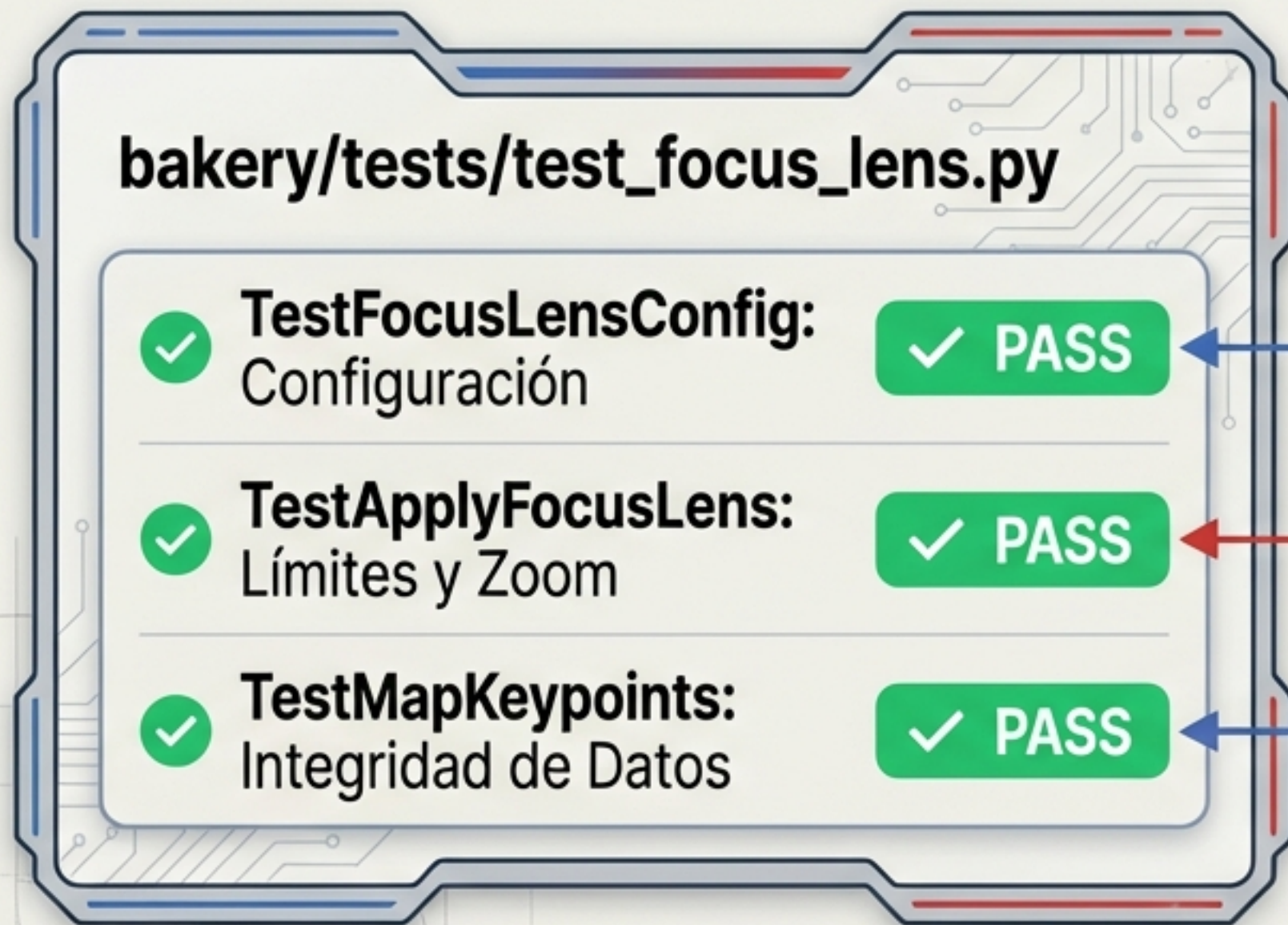
→ **Memoria (RAM):** La estrategia **Zoom** es la más costosa, ya que requiere asignar memoria para el frame escalado antes del recorte. El mapeo de máscaras crea matrices grandes ($O(N \times H \times W)$).

→ **Cómputo (CPU/GPU):** El impacto es despreciable comparado con la inferencia del modelo. Las transformaciones utilizan **NumPy vectorizado** para procesar lotes de coordenadas instantáneamente.

→ **Trade-off Operativo:** Se gana **precisión extrema** en el objeto a cambio de perder la **visión periférica** fuera del recorte.

Validación y Testing (BDD)

Garantizando robustez mediante ``bakery/tests/test_focus_lens.py``.



Coverage Points

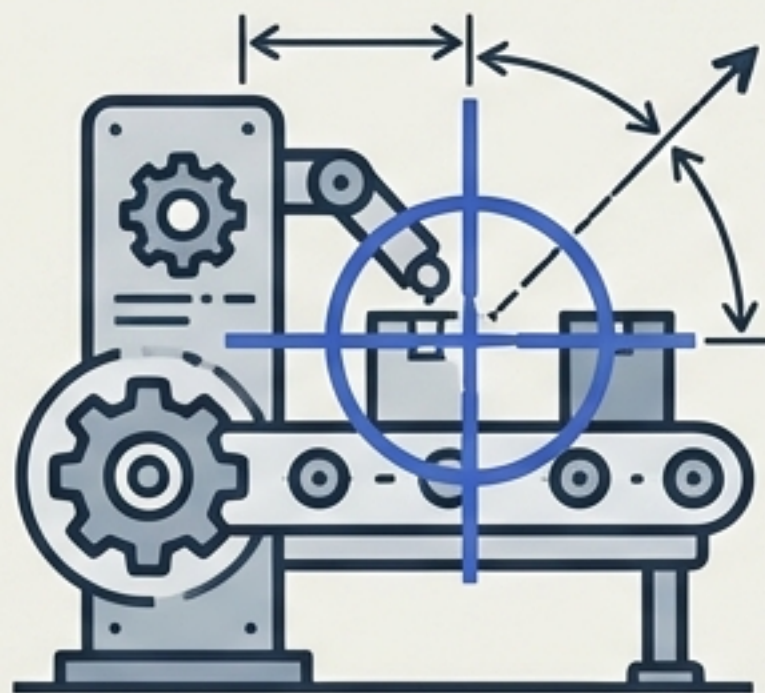
- **Configuración:** Rechazo inmediato de tamaños inválidos o negativos.
- **Límites (Boundaries):** Pruebas exhaustivas de 'Clamping' para recortes que salen del frame.
- **Integridad de Datos:** Verificación de que los keypoints invisibles (0,0) permanecen inalterados tras el mapeo.
- **Compatibilidad:** Tests de conversión de tipos para el anotador.

Casos de Uso Comunes



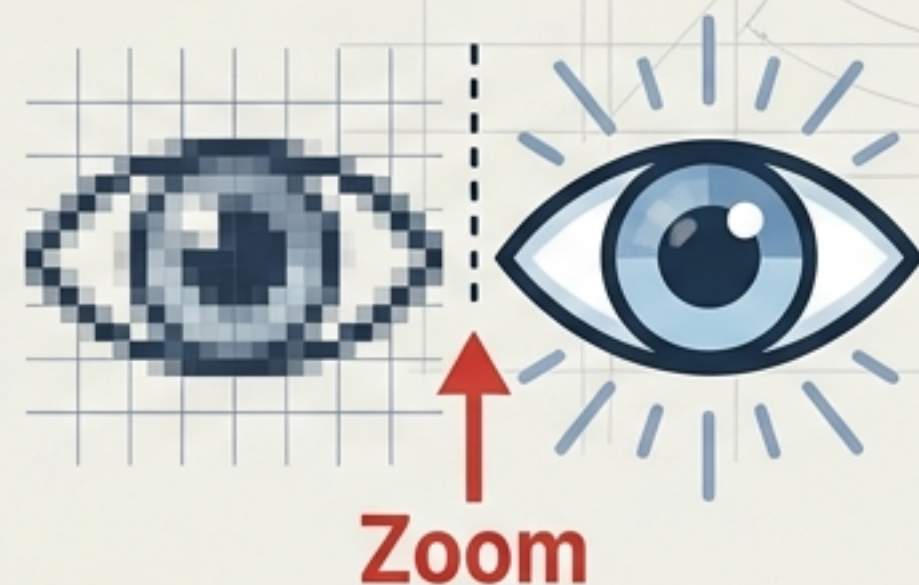
Tracking Dinámico

Tracking de Personas:
Configuración centrada (``focus_x=None``) para seguir sujetos en movimiento o centrar la atención.



Monitoreo Fijo

Zona Fija Industrial:
Coordenadas estáticas para monitorear válvulas, puertas o maquinaria específica.



Mejora de Calidad

Video de Baja Calidad:
Uso de estrategia **Zoom** para generar una "super-resolución" artificial antes de la inferencia.



Conclusión: Precisión sin Re-entrenamiento

- **Modular:** Se integra en el pipeline existente sin fricción.
- **Seguro:** Tipado estricto, inmutabilidad y validación de bordes.
- **Efectivo:** Transforma detecciones locales en coordenadas globales transparentemente.

Referencia: `bakery/core/entities/focus_lens_config.py`